Linked list

Linked List

- A linked list is a data structure used to store a collection of elements.
- Each element, called a node, contains the data and a reference (or pointer) to the next node in the sequence.
- This structure allows for efficient insertions and deletions since you only need to update the pointers.

Reference: Dev Frank, **C-LINKED LIST (Singly Linked List)** Medium

https://medium.com/@Dev_Frank/c-linked-list-singly-linked-list-12aecb168834

Types of Linked Lists

- **Singly Linked List**: Each node points to the next node. The last node points to **NULL**.
- **Doubly Linked List**: Each node points to both the next and the previous node.
- **Circular Linked List**: The last node points back to the first node, forming a circle.

Node Definition

1. **Node:** A node is the basic building block of a linked list. Each node contains data and a pointer to the next node.
   - **Data**: This is the value or information the node holds.
   - **Pointer**: This is like an address that tells you where the next node in the list is.

2. **Head:** This a pointer or reference to the first node in the list.

   - It serves as the starting point for any operations that involve traversing or manipulating the list.
   - Without the head, it would be impossible to access the rest of the nodes in the list.

```
// Define the node structure
struct Node {
    int data;           // Data part
    struct Node* next; // Pointer to the next node
};
```

1. Creation of a Node:

- You need a function to create and initialize a new node.
- Allocating memory for a new node.
- Setting the node's data and initializing the **next** pointer to **NULL**.

```c
#include <stdio.h>
#include <stdlib.h>

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

2. Inserting Nodes

Nodes can be inserted at the beginning, at the end, or at a specific position.

— **insertAtBeginning** function creates a new node, sets its **next** pointer to the current head, and updates the head to the new node.

```
void insertAtBeginning(struct Node** headRef, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *headRef;
    *headRef = newNode;
}
```

2. Inserting Nodes

— **insertAtEnd** function creates a new node, sets its **next** pointer to the current head, and updates the head to the new node.

```
void insertAtEnd(struct Node** headRef, int data) {
    struct Node* newNode = createNode(data);
    if (*headRef == NULL) {
        *headRef = newNode;
    } else {
        struct Node* temp = *headRef;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

3. Deletion :

Deletion can occur at various positions in the list: at the beginning, at the end, or at a specific position.

Delete from the Beginning

To delete the first node, you simply move the head pointer to the next node.

```c
void deleteFromBeginning(struct Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;
    *headRef = (*headRef)->next;
    free(temp);
}
```

- Checks if the list is empty. If it is, it prints a message.
- If not, it updates the head to point to the next node and frees the memory of the original first node.

3. Deletion :

Delete from the End : To delete the last node, you need to traverse the list to find the second last node and update its **next** pointer to **NULL**.

```c
void deleteFromEnd(struct Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    if ((*headRef)->next == NULL) {
        free(*headRef);
        *headRef = NULL;
        return;
    }
    struct Node* temp = *headRef;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

- Checks if the list is empty. If it is, it prints a message.
- If the list has only one node, it frees it and sets the head to **NULL**.
- Otherwise, it traverses the list to find the second last node, updates its **next** pointer to **NULL**, and frees the last node.

3. Deletion :

Delete from a Specific Position

To delete a node at a specific position, you need to find the node just before that position and update its next pointer

- Checks if the list is empty. If it is, it prints a message.
- If the position is **0**, it updates the head to point to the second node and frees the first node.
- Otherwise, it traverses the list to find the node just before the position, updates its **next** pointer to skip the node to be deleted, and frees the node.

```c
void deleteFromPosition(struct Node** headRef, int position) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;

    if (position == 0) {
        *headRef = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position does not exist\n");
        return;
    }

    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}
```

Main function

```c
int main() {
    struct Node* head = NULL;

    // Insert nodes
    insertAtBeginning(&head, 10);
    insertAtBeginning(&head, 20);
    insertAtBeginning(&head, 30);

    // Print list
    printf("Initial list: ");
    printList(head);

    // Delete from beginning
    deleteFromBeginning(&head);
    printf("After deleting from beginning: ");
    printList(head);

    // Delete from end
    deleteFromEnd(&head);
    printf("After deleting from end: ");
    printList(head);

    // Delete from a specific position
    insertAtBeginning(&head, 40);
    insertAtBeginning(&head, 50);
    deleteFromPosition(&head, 1);
    printf("After deleting from position 1: ");
    printList(head);

    return 0;
}
```

4. Traversing the List :

Starting from the head, you follow the pointers from one node to the next until you reach the end.

- Starts at the head node.
- Iterates through the list using a temporary pointer (**temp**).
- Prints the data of each node
- Stops when **temp** becomes **NULL**, indicating the end of the list.

```c
// Function to traverse the list and print the elements
void traverseList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Doubly Linked List and operations

- An integer data value.
- A pointer to the next node.
- A pointer to the previous node.

```c
typedef struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
} Node;
```

- Insert at the Beginning
- Insert at the End
- Delete a Node
- Display List

https://medium.com/@Dev_Frank/c-linked-list-doubly-linked-list-561851cd732a

Creation of a Node

```
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}
```

Insertion :: At Front

```
void insertAtFront(Node** head, int data) {
    Node* newNode = createNode(data);
    newNode->next = *head;
    if (*head != NULL) {
        (*head)->prev = newNode;
    }
    *head = newNode;
}
```

Insertion :: At End

```
void insertAtEnd(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}
```

Inserting a Node After a Given Node:

```
void insertAfter(Node* prevNode, int data) {
    if (prevNode == NULL) {
        printf("The given previous node cannot be NULL\n");
        return;
    }

    Node* newNode = createNode(data);
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if (prevNode->next != NULL) {
        prevNode->next->prev = newNode;
    }

    prevNode->next = newNode;
}
```

Inserting a Node Before a Given Node:

```
void insertBefore(Node** head, Node* nextNode, int data) {
    if (nextNode == NULL) {
        printf("The given next node cannot be NULL\n");
        return;
    }

    Node* newNode = createNode(data);
    newNode->prev = nextNode->prev;
    newNode->next = nextNode;

    if (nextNode->prev != NULL) {
        nextNode->prev->next = newNode;
    } else {
        // If the new node is inserted before the head node
        *head = newNode;
    }

    nextNode->prev = newNode;
}
```

Traverse

```
void traverseForward(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```
void traverseBackward(Node* head) {
    if (head == NULL) {
        return;
    }

    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}
```

Deletion:: From the Front: Remove a node from the beginning of the list

```
void deleteFromFront(Node** head) {
    if (*head == NULL) {
        printf("The list is already empty\n");
        return;
    }

    Node* temp = *head;
    *head = (*head)->next;

    if (*head != NULL) {
        (*head)->prev = NULL;
    }

    free(temp);
}
```

Deletion::From the End: Remove a node from the end of the list.

```
void deleteFromEnd(Node** head) {
    if (*head == NULL) {
        printf("The list is already empty\n");
        return;
    }

    Node* temp = *head;

    // Traverse to the last node
    while (temp->next != NULL) {
        temp = temp->next;
    }

    // If there's only one node in the list
    if (temp->prev == NULL) {
        *head = NULL;
    } else {
        temp->prev->next = NULL;
    }

    free(temp);
}
```

Deletion::Given Node: Remove a specific node from the list.

```
void deleteGivenNode(Node** head, Node* del) {
    if (*head == NULL || del == NULL) {
        printf("The list or the given node is NULL\n");
        return;
    }

    if (*head == del) {
        *head = del->next;
    }

    if (del->next != NULL) {
        del->next->prev = del->prev;
    }

    if (del->prev != NULL) {
        del->prev->next = del->next;
    }

    free(del);
}
```

Traversal::Forward Traversal: Visit each node starting from the head and moving to the tail.

```
void traverseForward(Node* head) {
    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

Traversal::Backward Traversal: Visit each node starting from the tail and moving to the head.

```
void traverseBackward(Node* head) {
    Node* temp = head;
    if (temp == NULL) {
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
    }
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->prev;
    }
    printf("\n");
}
```

Search

```
Node* searchNode(Node* head, int target) {
    Node* current = head;
    while (current != NULL) {
        if (current->data == target) {
            return current;
        }
        current = current->next;
    }
    return NULL; // Node not found
}
```

Update

```
void updateNode(Node* head, int oldData, int newData) {
    Node* current = head;
    while (current != NULL) {
        if (current->data == oldData) {
            current->data = newData;
            return;
        }
        current = current->next;
    }
    printf("Node with data %d not found.\n", oldData);
}
```

Reverse

```
void reverse(Node** head) {
    Node* temp = NULL;
    Node* current = *head;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }

    if (temp != NULL) {
        *head = temp->prev;
    }
}
```

Circular Linked List and operations

Singly Circular Linked List : The last node in the single linked list points back to the **first node**, forming a loop.

Doubly Circular Linked List: Each node has two pointers (next and previous). The last node points forward to the first node **and** the first node points backward to the last node.

Circular Linked List:: Node definition

```
// Define the node structure
struct Node {
    int data;           // Data part
    struct Node* next; // Pointer to the next node
};
```

Circular Linked List:: Node creation

```c
#include <stdio.h>
#include <stdlib.h>

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

Circular Linked List:: insertion at beginning

```
void insertAtBeginning(struct Node** headRef, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *headRef;
    *headRef = newNode;
}
```

This is for Single linked list

Can you say where you need to change in the above code to make it circular linked list?

```c
struct Node* insertAtBeginning(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    // If list is empty
    if (head == NULL) {
        newNode->next = newNode;   // Points to itself
        head = newNode;
    } else {
        struct Node* temp = head;

        // Traverse to the last node
        while (temp->next != head) {
            temp = temp->next;
        }

        newNode->next = head;  // New node points to old head
        temp->next = newNode;  // Last node points to new node
        head = newNode;        // Update head
    }

    return head;
}
```

Circular Linked List:: insertion at beginning

Circular Linked List:: insertion at end

```
void insertAtEnd(struct Node** headRef, int data) {
    struct Node* newNode = createNode(data);
    if (*headRef == NULL) {
        *headRef = newNode;
    } else {
        struct Node* temp = *headRef;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}
```

This is for singled linked list.

Think how to make it for circular linked list?

```c
struct Node* insertAtEnd(struct Node* head, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    // If list is empty
    if (head == NULL) {
        newNode->next = newNode;  // Point to itself
        return newNode;
    }

    struct Node* temp = head;

    // Traverse to the last node
    while (temp->next != head) {
        temp = temp->next;
    }

    temp->next = newNode;   // Last node points to new node
    newNode->next = head;   // New node points to head

    return head;  // Head remains unchanged
}
```

Circular Linked List:: insertion at end

```c
void deleteFromBeginning(struct Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;
    *headRef = (*headRef)->next;
    free(temp);
}
```

single Linked List:: Delete from beginning

```c
struct Node* deleteFromBeginning(struct Node* head) {
    // If list is empty
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If only one node exists
    if (head->next == head) {
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* last = head;

    // Find the last node
    while (last->next != head) {
        last = last->next;
    }

    head = head->next;      // Move head to next node
    last->next = head;      // Last node points to new head
    free(temp);             // Free old head

    return head;
}
```

Circular Linked List:: delete from beginning

```c
void deleteFromEnd(struct Node** headRef) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    if ((*headRef)->next == NULL) {
        free(*headRef);
        *headRef = NULL;
        return;
    }
    struct Node* temp = *headRef;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}
```

Single Linked List:: Delete from end

```c
struct Node* deleteFromEnd(struct Node* head) {
    // If list is empty
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If only one node
    if (head->next == head) {
        free(head);
        return NULL;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    // Traverse to last node
    while (temp->next != head) {
        prev = temp;
        temp = temp->next;
    }

    // temp = last node
    // prev = second last node
    prev->next = head;  // Make second last point to head
    free(temp);         // Delete last node

    return head;        // Head remains unchanged
}
```

Circular Linked List:: delete from end

```c
void deleteFromPosition(struct Node** headRef, int position) {
    if (*headRef == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node* temp = *headRef;

    if (position == 0) {
        *headRef = temp->next;
        free(temp);
        return;
    }

    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position does not exist\n");
        return;
    }

    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}
```

Single Linked List:: Delete from a specific position

Circular Linked List:: delete from a specific poisition

```c
struct Node* deleteFromPosition(struct Node* head, int position) {
    if (head == NULL) {
        printf("List is empty\n");
        return NULL;
    }

    // If position is 1 → delete from beginning
    if (position == 1) {
        // Only one node
        if (head->next == head) {
            free(head);
            return NULL;
        }

        struct Node* last = head;
        while (last->next != head) {
            last = last->next;
        }

        struct Node* temp = head;
        head = head->next;
        last->next = head;
        free(temp);

        return head;
    }
```

```c
    // Delete from position other than 1
    struct Node* temp = head;
    struct Node* prev = NULL;
    int count = 1;

    while (count < position && temp->next != head) {
        prev = temp;
        temp = temp->next;
        count++;
    }

    // If position is invalid
    if (count != position) {
        printf("Invalid position\n");
        return head;
    }

    prev->next = temp->next;
    free(temp);

    return head;
}
```

Header List and Sentinel Node

Generalized Linked List

Skip List

Applications of Linked Lists

Polynomial manipulation using Linked List

Implementation of other data structures using Linked List

Revision/Problem discussion on Linked Lists